

Rapport de Travaux Pratiques – Séance 7

Sérialisation, Marshalling et Sécurisation des Flux

Filière Cybersécurité – Cycle Ingénieur 1A
EUROMED Fès

14 juin 2026

Table des matières

1	Introduction	2
2	Étude comparative des formats de sérialisation	2
3	Contre-mesures de Cybersécurité Implémentées	2
3.1	Neutralisation des Attaques XXE	2
3.2	Sécurisation cryptographique de Pickle	2
4	Architecture du Contrat de Données (Protobuf)	2
5	Conclusion	3

1 Introduction

La sérialisation est le processus consistant à convertir un objet en mémoire en un flux d'octets transmissible sur le réseau. Ce TP étudie les failles de sécurité majeures liées à ce processus (notamment les vulnérabilités de désérialisation non sécurisée et les attaques XXE) et met en œuvre des protocoles d'échange sécurisés basés sur des schémas stricts.

2 Étude comparative des formats de sérialisation

Trois formats ont été analysés et sécurisés durant cette séance :

1. **JSON / Protobuf** : Passage d'un format texte à un format binaire strict régi par un schéma fortement typé (`.proto`).
2. **XML** : Analyse des risques d'inclusions d'entités externes (XXE).
3. **Pickle (Python)** : Atténuation des risques d'exécution de code arbitraire (RCE).

3 Contre-mesures de Cybersécurité Implémentées

3.1 Neutralisation des Attaques XXE

L'utilisation de parseurs XML natifs expose à la lecture de fichiers locaux (*Arbitrary File Reading*) via la déclaration d'entités système. La bibliothèque `defusedxml` a été déployée pour interdire les DTD (Document Type Definitions) malveillants.

3.2 Sécurisation cryptographique de Pickle

Le module `pickle` est intrinsèquement dangereux car la méthode `__reduce__()` permet d'instancier des objets système malveillants (ex : `os.system`). Pour garantir l'intégrité du flux, nous implémentons une signature **HMAC-SHA256** avec clé secrète partagée.

Le processus de vérification s'assure que :

$$\text{HMAC}_{\text{calculé}} \equiv \text{HMAC}_{\text{reçu}} \quad (1)$$

Cette comparaison est réalisée en temps constant via `hmac.compare_digest` pour contrecarrer les attaques par canaux auxiliaires temporels (*Timing Attacks*).

4 Architecture du Contrat de Données (Protobuf)

Voici la structure du fichier de contrat binaire `document.proto` :

Listing 1 – Schéma Protobuf sécurisé

```
syntax = "proto3";

message Document {
    string title = 1;
    string content = 2;
    string visibility = 3;
}
```

5 Conclusion

La sérialisation basée sur un schéma explicite (comme Protocol Buffers) s'impose comme la solution la plus robuste en cybersécurité. Elle élimine nativement la possibilité d'injecter des comportements exécutables au sein des payloads de données.